

Authentication

Section Review

Authentication

- Authentication is the act of logging the user into the system.
- Token-based authentication involves the client/user supplying a token (i.e., JWT) on every request after the initial login.
- The server parses the token/JWT on every request.

Password Hashing

- Hashing is a one-way data transformation that converts a starting piece of text into a new string of characters (called the hash).
- Never store user passwords as plain text in the database. Store the hashed versions.
- Password hashing is intentionally expensive, which helps slow down hackers.

User Registration and Login

- When a user creates an account, save their credentials (i.e. email, username) and the hashed password.
- When a user logs in, verify their submitted password against the hashed password in the database.

JWT Tokens

- A **JSON Web Token** (JWT) is a token/string that encodes information in a specific format.
- A JWT consists of a **header** (metadata), a **payload** (data/information about the user), and a **signature** (proof the token has not been tampered with).

Secret Keys

- The **secret key** generates a distinct JWT.
- It influences the signature, the third segment of the JWT.
- The secret key for your production app should be secure and only shared with trusted developers.

Environment Variables

- An environment variable is a value from the “outside world” (outside of the Python code/application)
- A common pattern in local development is to create an **.env** file that stores environment variables.
- In production, the developer supplies the environment variables through the hosting platform’s (i.e. AWS) interface.

Pydantic Settings

- **pydantic-settings** is a Python library that reads environment variables into a **BaseSettings** model, a subclass of **BaseModel**.
- Pydantic can perform validations and transformations on the imported values.
- Pydantic can also supply fallback values for environment variables.

Back to Authentication

- When the user successfully logs in, the server generates a JWT and sends it in the response.
- The client app's responsibility is to send the JWT on all follow-up server requests. The JWT serves as proof of authentication.
- JWTs usually have a short expiration time (~10 minutes).

Decoding the Token

- On the server, the secret key is used to verify and decode the JWT.
- It is cryptographically difficult for a hacker to generate a valid JWT if they do not have the secret key (VERY large number of combinations).
- The short expiration window limits how long a stolen JWT remains valid.

The OAuth2PasswordBearer Class

- The **OAuth2PasswordBearer** class from **fastapi.security** adds an **Authorize** button to the Swagger documentation.
- Once the user logs in, Swagger supplies the JWT via the **Authorization** header on every subsequent request.
- Pass the **tokenUrl** parameter the authentication endpoint that returns a token.

The `get_current_user` Dependency

- We defined a `get_current_user` dependency function that validated a user is authenticated and a `CurrentUser` type.
- The PyJWT library validates that a token is valid and unexpired.
- We also validated the presence of an `email` claim and the presence of the `User` in the database.

Protecting Routes

- Adding the **CurrentUser** type as a route parameter requires authentication to access the route.
- If the route handler does not need the **current_user** object, add **Depends(get_current_user)** to the route's **dependencies** list or the router's **dependencies** list.

Refresh Tokens I

- A **refresh token** grants permission to generate a new JWT access token.
- In our application, the refresh token was itself a JWT signed with the same secret key but with a different type/purpose.

Refresh Tokens II

- When the access token expires, the client sends the refresh token to an endpoint to retrieve a new access token.
- The benefit is that the user does not have to login as frequently.
- The refresh token has a longer expiration timeframe (~1 week).

Logout

- JWTs are a stateless authentication solution.
- The client app can delete the locally stored JWT access token so the user returns to an unauthenticated state and has to log in again.
- For better security, the server will need to introduce stateful solutions.

Password Reset I

- The unauthenticated user hits an endpoint that sends an email to the specified address with a one-time-use token.
- The token grants permission to update the password.
- The user clicks a link in the email and accesses an app page where they submit their new password.

Password Reset II

- The app submits the token (email -> query params -> endpoint) to a second endpoint.
- The endpoint verifies the token is valid, not expired, not used, etc.
- The endpoint updates the user's password. Make sure to hash it!